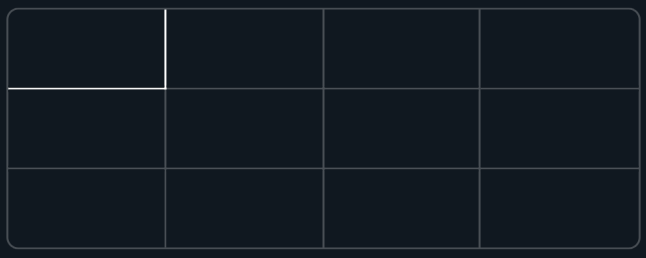
HTML & CSS QUICK REFERENCE

Theessentialguidetostructuring,stylingandbuilding modern websites.





HTML · CSS · LAYOUT · RESPONSIVE DESIGN

Acurated, practicalhandbookfor finding a tag, property, layout technique or code patternfast whilebuildingasite. For beginners, students, aspiring frontend developers anddesignersmoving into code.

How to use this guide

Scan the page titles, jump to the topic, copy the pattern. Page 10 holds the cheat sheet and references.

HTML Foundations: structure first, style second

HTML describes structureandmeaning;CSScontrolspresentation. The browser parses both into a page.

HTML + CSS Structure + style	→ Browser Parses and renders	→ Webpage Visible result

● HTML / index.html

```
<!DOCTYPE html> <html lang="en">
<head><meta charset="UTF-8">
<meta name="viewport"
content="width=device-width, initial-scale=1">
<title>My first site</title>
<link rel="stylesheet" href="styles.css">
</head><body><main>
<h1>Welcome</h1><p>Build something useful.</p>
<a href="about.html">About this site</a>
</main></body></html>
```

`head` holds metadata; `body` holds content. Nest elements and use attributes such as `href`, `src` and `lang`. Use headings for hierarchy, `p` for paragraphs and `img` with meaningful `alt` text. Comments use `<!-- note -->`.

HTML Elements, Semantics and Forms

Category	Elements
Text	h1–h6, p, span, strong, em, br, hr, blockquote
Links / media	a, img, video, audio, source
Lists	ul, ol, li; dl, dt, dd for descriptions
Tables	table, thead, tbody, tr, th, td
Semantics	header, nav, main, section, article, aside, footer

Forms: form, label, input, textarea, select, option, button. **Types:** text, email, password, number, checkbox, radio, date, file, submit, search.

Attributes: name, value, placeholder, required, disabled, readonly, min, max. Name/value submit data; placeholder gives hints. Disabled controls are not submitted.

Semantic structure

header → nav

main

article + aside

footer

● HTML / labelled control

```
<form action="/subscribe" method="post">
<label for="email">Email</label>
<input id="email" name="email" type="email" required>
<button type="submit">Subscribe</button>
</form>
```

TIP: Write meaningful alt text and always label form controls.

CSS Fundamentals and Selectors

`selector { property:value;}` sets declarations;/*note*/isacomment. Use external CSS,internal `<style>`,oraninline style attribute.

Selector	Selects

`p` / `*`
`.card` `#header`
`h1, p`
	Paragraphs / every element
	A class / a unique ID
	Both types, grouped

`.card p`
`.card > p`
`input[type="email"]`
	Paragraph descendants
	Direct child paragraphs
	Matching attribute values

`a:hover` / `input:focus`
`li:first-child`
`li:nth-child(2n)`
	Hovered link / focused control First-child list item
	Even-positioned list items

`.card::before` / `.card::after` `input::placeholder`	Generated parts; usually need content Placeholder text

Resolve competing rules

The cascade considers origin, importance and layers before specificity. In the same context, IDs outrank classes, attributes and pseudo-classes, which outrank types and pseudo-elements; ties follow source order. Color inherits from a parent; margin does not.

TIP · Pseudo-classes select a state (`:hover`); pseudo-elements style a part (`::before`).

CSS Properties and the Box Model

BOXMODEL · outsidetoinside

margin	border	padding	content

Category	Common properties
Color / fill	color, background, background-color, background-image, background-size, opacity
Typography	font-family, font-size, font-weight, line-height, letter-spacing, text-align, text-decoration, text-transform
Spacing	margin, padding, gap, row-gap, column-gap
Edges	border, border-radius, box-shadow, outline
Dimensions	width, height, min-width, max-width, min-height, max-height, box-sizing, overflow

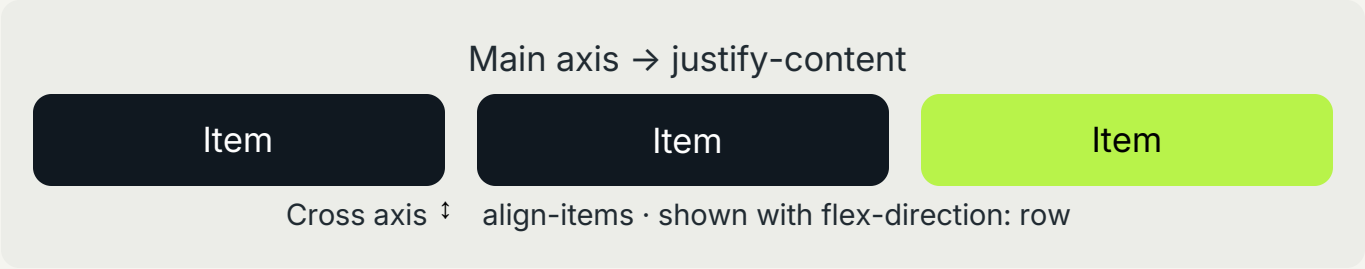
`content-box`: width and height apply to content; padding and border add size.
`border-box`: width and height include padding and border. Neither includes margin.

● CSS / predictable sizing

```
*, *::before, *::after { box-sizing: border-box; }
```

TIP · Use `*{box-sizing: border-box; }` for predictable layouts; keep keyboard outlines visible.

Flexbox and Positioning



Flexbox property	Job
display: flex; flex-direction	Enable flex; choose row or column
justify-content; align-items	Align on main / cross axis
flex-wrap; align-content	Wrap; distribute multiple lines
flex-grow; flex-shrink	Share extra space; absorb shortage
flex-basis; gap; align-self	Initial size; spacing; item override

● CSS / center both ways

```
.center{display:flex;
justify-content:center;
align-items:center;
min-height:12rem;}
```

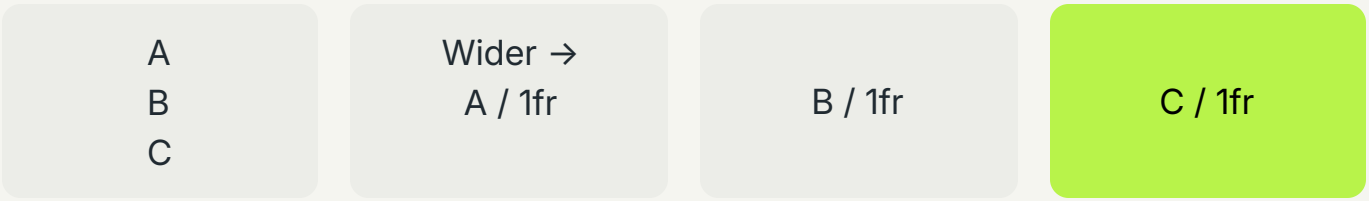
Positioning

static follows normal flow. **relative** keeps its space while allowing offsets. **absolute** leaves flow and uses its containing block. **fixed** usually anchors to the viewport. **sticky** stays in flow, then sticks at an inset such as top: 0. **z-index** controls stacking within stacking contexts.

TIP · Changing flex-direction changes which axis each alignment property uses.

CSS Grid and Responsive Design

Grid syntax	Job
display: grid; grid-template-columns / rows	Enable layout; define column / row tracks
grid-column / grid-row; grid-area	Place or span an item; name an area
gap; repeat(); minmax(); fr	Gutters; repetition; bounds; free-space share




```
.grid { display: grid; gap: 1rem;
 grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
img { max-width: 100%; height: auto; }
@media (min-width: 48rem) { .sidebar { display: block; } }
```

Start mobile-first with flexible widths and the viewport meta tag from page 2. Grid adds columns as space allows; its minimum track must fit the smallest container.

TIP · Choose breakpoints for layout changes, not device names.

Modern CSS Techniques and Units

● CSS / syntax sampler

```
:root {
  --primary-color: #B8F34A;
}
color: var(--primary-color);
width: min(100%, 60rem);
min-height: max(10rem, 20vh);
padding: calc(1rem + 1vw);
font-size: clamp(1rem, 2vw, 1.5rem);
background: linear-gradient(90deg,
#101820, #334);
transition: transform 200ms ease;
transform: translateY(-2px);
object-fit: cover;
filter: contrast(1.1);
backdrop-filter: blur(8px);
animation: appear 300ms ease;
```

Unit	Relative to
px	A CSS reference pixel, not necessarily a device pixel
%	A property-dependent reference, often the containing block
em / rem	Element font size / root font size
vw / vh	1% of viewport width / height
fr	A share of available space in Grid tracks
ch	The font's "0" advance width, useful for text measures

Custom properties reuse values; clamp() sets a minimum, preferred value and maximum. object-fit needs a sized media box; backdrop-filter needs transparency to reveal its effect.

Respect reduced-motion preferences. Newer features can have browser-support considerations; check MDN compatibility tables.

Practical Code Examples

Navigation bar

```
<style>nav{display:flex;gap:1rem;padding:1rem}
nav a:hover,nav a:focus{text-decoration:none}</style>
<nav aria-label="Primary"><a href="/">Home</a>
<a href="/about">About</a></nav>
```

Preview: Home About — Flexbox spaces links; focus keeps its native outline.

Product card

```
<style>
.p{padding:1rem;
border:1px solid #87939e;
border-radius:8px; box-
shadow:0 2px 8px #0002} .p
h2{font:700 1.2rem sans-serif}
</style>
<article class="p">
<img src="notebook.jpg"
alt="Field notebook">
<h2>Field notebook</h2>
<p>Plain paper.</p></article>
```

Preview: Field notebook · Plain paper. Padding separates content from its border; use descriptive alt text.

Responsive card grid

```
<style>
.cards{display:grid;gap:1rem}
@media(min-width:40rem){
.cards{grid-template-columns:
repeat(auto-fit,
minmax(10rem,1fr))}}
</style>
<div class="cards">
<p>A</p><p>B</p><p>C</p>
</div>
```

Preview: A | B | C
One column first; wider containers gain flexible tracks.

Cheat Sheet, Best Practices and References

Task	Reach for
HTML	header, nav, main, section, article, footer; h1–h6, p, a, img, ul/li, button
Select	class, #id, [attribute], :focus, ::before
Layout	display: flex; gap; justify-content; align-items; display: grid; grid-template-columns
Size	rem for type/spacing; % for flexible size; fr for tracks
Adapt	`@media (min-width: 48rem) { … }`

Build accessibly, then test

Use semantic HTML, logical headings, meaningful alt text and labelled form controls. Reuse classes; avoid unnecessary `!important`. Ensure readable contrast and visible keyboard focus. Test responsive layouts at different widths.

Glossar y

DOM / CSSOM: document / style models. **Semantic HTML:** meaningful elements. **Specificity:** selector weight in the cascade. **Viewport:** visible layout area. **Breakpoint:** layout-change condition. **Responsive design:** layouts adapt to available space.

References

MDN Web Docs · developer.mozilla.org
WHATWG HTML Standard · html.spec.whatwg.org
W3C CSS resources · www.w3.org/Style/CSS